

Cheat Sheet For Simple WebGL2 Drawing

1. Set Up WebGL2 Context

- `const canvas = document.getElementById('canvas-id');`
 - Retrieve the <canvas> element.
 - `const gl = canvas.getContext('webgl2');`
 - Get the WebGL2 rendering context.
-

2. Define Shaders (Vertex and Fragment) - ECMAScript 300 Style

Vertex Shader (ES3)

The vertex shader processes each vertex (no precision and simple types):

```
#version 300 es
precision mediump float;

in vec4 a_position; // Vertex position

void main() {
    gl_Position = a_position; // Pass position to the pipeline
}
```

Fragment Shader (ES3)

The fragment shader defines the pixel color :

```
#version 300 es
precision mediump float;

out vec4 color;
void main() {
    color = vec4(1.0, 0.0, 0.0, 1.0); // Red color
}
```

3. Create Shader Program

- **Create Shader and Program:**
- `const vertexShader = createShader(gl, gl.VERTEX_SHADER, vertexShaderSource);`
- `const fragmentShader = createShader(gl, gl.FRAGMENT_SHADER, fragmentShaderSource);`
- `const program = createProgram(gl, vertexShader, fragmentShader);`

4. Create Buffers

- **Create and Bind a Buffer for Vertex Data:**
 - `const vertexBuffer = gl.createBuffer();`
 - `gl.bindBuffer(gl.ARRAY_BUFFER, vertexBuffer);`
 - **Upload Vertex Data:**
 - `const vertices = [`
 - `0.0, 0.5, 0.0, // Vertex 1`
 - `-0.5, -0.5, 0.0, // Vertex 2`
 - `0.5, -0.5, 0.0 // Vertex 3`
 - `];`
 - `gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices), gl.STATIC_DRAW);`
-

5. Set Up Attribute Pointers (For Vertex Data)

- **Get Attribute Location:**
 - `const positionAttribLocation = gl.getAttribLocation(program, "a_position");`
 - **Specify Data Layout:**
 - `gl.vertexAttribPointer(positionAttribLocation, 3, gl.FLOAT, false, 0, 0);`
 - **Enable Attribute Array:**
 - `gl.enableVertexAttribArray(positionAttribLocation);`
-

6. Set Up and Use Shader Program

- **Use the Shader Program:**
 - `gl.useProgram(program);`
-

7. Set Clear Color (Optional)

- **Define the Background Color (e.g., black):**
 - `gl.clearColor(0.0, 0.0, 0.0, 1.0);`
 - **Clear the Canvas:**
 - `gl.clear(gl.COLOR_BUFFER_BIT);`
-

8. Draw the Shape

- **Draw Using `gl.drawArrays` (For Non-Indexed Drawing):**
- `gl.drawArrays(gl.TRIANGLES, 0, vertices.length / 3);`
 - `gl.TRIANGLES`: Specifies drawing triangles.
 - `0`: Start drawing from the first vertex.

- `vertices.length / 3`: The number of vertices (in this case, we have 3).
 - **Or, Draw Using `gl.drawElements` (For Indexed Drawing):**
 - If you are using an index buffer:
 - `const indices = [0, 1, 2]; // Indices for the vertices`
 - `const indexBuffer = gl.createBuffer();`
 - `gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);`
 - `gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint16Array(indices), gl.STATIC_DRAW);`
 -
 - `gl.drawElements(gl.TRIANGLES, indices.length, gl.UNSIGNED_SHORT, 0);`
-

9. Cleanup (Optional, but Good Practice)

- **Delete Buffers and Program:**
 - `gl.deleteBuffer(vertexBuffer);`
 - `gl.deleteProgram(program);`
 - `gl.deleteShader(vertexShader);`
 - `gl.deleteShader(fragmentShader);`
-

Order of Methods for Drawing a Shape (Summary)

1. **Set up WebGL2 context** (`getContext('webgl2')`)
 2. **Define and create shaders** (vertex and fragment shaders in ES3)
 3. **Create and bind buffers** (for vertex data)
 4. **Upload data to buffers** (vertex data)
 5. **Set up attribute pointers** (for vertex attributes)
 6. **Use the shader program** (`gl.useProgram`)
 7. **Set the clear color** (optional, before drawing)
 8. **Clear the canvas** (`gl.clear`)
 9. **Draw the shape** (`gl.drawArrays` OR `gl.drawElements`)
 10. **Clean up** (delete buffers and programs when done)
-

Uniforms in Shaders

Uniforms are values that stay constant across the entire draw call, and they are usually used for things like transformation matrices, colors, lighting parameters, and more. They can be passed from JavaScript to the shader program.

1. Defining Uniforms in Shaders

Vertex Shader (ES3) with Uniform Example:

In the vertex shader, uniforms are typically used for transformation matrices (like model, view, projection matrices).

```
#version 300 es
precision mediump float;

in vec4 a_position; // Vertex position
uniform mat4 u_modelViewMatrix; // Model-view matrix
uniform mat4 u_projectionMatrix; // Projection matrix

void main() {
    // Apply model-view and projection transformations to the vertex position
    gl_Position = u_projectionMatrix * u_modelViewMatrix * a_position;
}
```

- **uniform mat4 u_modelViewMatrix;** This is a 4x4 matrix that transforms the vertex position to camera space.
- **uniform mat4 u_projectionMatrix;** This matrix handles the perspective projection (e.g., perspective or orthographic).

Fragment Shader (ES3) with Uniform Example:

Uniforms in the fragment shader can be used for colors, lighting parameters, or any other properties that remain constant.

```
#version 300 es
precision mediump float;

out vec4 color;
uniform vec4 u_color; // A color (RGBA)
void main() {
    // Set the fragment color to the uniform value
    color = u_color;
}
```

- **uniform vec4 u_color;** A uniform for the color of the shape (e.g., RGBA values).

2. Setting Up and Using Uniforms in JavaScript

After you've declared the uniforms in your shader, you can set them from JavaScript using `gl.getUniformLocation` and `gl.uniform*` methods. These methods allow you to pass values to the uniforms that you defined in your shaders.

JavaScript Code to Set Uniforms

After compiling and linking your shaders into a program, you can retrieve the uniform locations and set their values before drawing the shape.

1. **Retrieve the Uniform Location:**
 2. `const modelViewMatrixUniformLocation = gl.getUniformLocation(program, "u_modelViewMatrix");`
 3. `const projectionMatrixUniformLocation = gl.getUniformLocation(program, "u_projectionMatrix");`
 4. `const colorUniformLocation = gl.getUniformLocation(program, "u_color");`
 5. **Set the Uniforms with Values:**
 - **For matrices (like model-view and projection matrices),** you will often use a library like `gl-matrix` to help create and manipulate matrices.
 6. `// Using gl-matrix for matrix manipulation`
 7. `const mat4 = glMatrix.mat4; // gl-matrix library`
 8. `const modelViewMatrix = mat4.create();`
 9. `const projectionMatrix = mat4.create();`
 - 10.
 11. `mat4.lookAt(modelViewMatrix, [0, 0, 5], [0, 0, 0], [0, 1, 0]); // Camera at (0,0,5), looking at origin`
 12. `mat4.perspective(projectionMatrix, Math.PI / 4, canvas.width / canvas.height, 0.1, 100.0); // Perspective projection`
 - 13.
 14. `// Set uniform values`
 15. `gl.uniformMatrix4fv(modelViewMatrixUniformLocation, false, modelViewMatrix);`
 16. `gl.uniformMatrix4fv(projectionMatrixUniformLocation, false, projectionMatrix);`
 - 17.
 18. `// Set color uniform (e.g., red)`
 19. `gl.uniform4fv(colorUniformLocation, [1.0, 0.0, 0.0, 1.0]); // RGBA Red color`
 - **For simple types (like color or floats),** you can use the appropriate `uniform*` method:
 - **For a `vec4` (color in this case):**
 - `gl.uniform4fv(colorUniformLocation, [1.0, 0.0, 0.0, 1.0]); // Red`
 - **For a float:**
 - `const someFloatLocation = gl.getUniformLocation(program, "u_someFloat");`
 - `gl.uniform1f(someFloatLocation, 1.0);`
-

3. Animation

If you want to animate something in JavaScript you need to change something over time and draw again.

```
1. requestAnimationFrame(drawScene);
2.
3. // Draw the scene.
4. function drawScene() {
5. // Every frame increase the rotation a little.
6.     rotation[1] += rotationSpeed / 60.0;
7.
8.     ...
9.     // Call drawScene again next frame
```

```
10.     requestAnimationFrame(drawScene);
11. }
```

4. 2D Matrices

We can multiply matrices together and apply all the transformations at once.

```
1. var m3 = {
2.   translation: function(tx, ty) {
3.     return [
4.       1, 0, 0,
5.       0, 1, 0,
6.       tx, ty, 1,
7.     ];
8.   },
9.
10.  rotation: function(angleInRadians) {
11.    var c = Math.cos(angleInRadians);
12.    var s = Math.sin(angleInRadians);
13.    return [
14.      c, -s, 0,
15.      s, c, 0,
16.      0, 0, 1,
17.    ];
18.  },
19.
20.  scaling: function(sx, sy) {
21.    return [
22.      sx, 0, 0,
23.      0, sy, 0,
24.      0, 0, 1,
25.    ];
26.  },
27. };
```

To revert angle in degree to radians:

```
let angle_radian = 2 * Math.PI / angle_degree;
```

5. Recap of Uniform Handling in Shaders

- **Define uniforms** in the GLSL shaders using `uniform <type> <name>;`.
 - **Retrieve uniform locations** in JavaScript using `gl.getUniformLocation(program, "uniformName")`.
 - **Set uniform values** using methods like `gl.uniform1f`, `gl.uniform4fv`, or `gl.uniformMatrix4fv` depending on the uniform type.
-

Additional Notes

- **Uniforms are constant across a draw call:** They don't change per vertex or per fragment. If you need values that change for each vertex or fragment, you should use **in** (for vertex data).
- **Matrix uniforms:** For matrices, you will typically use a library like gl-matrix for easier handling.